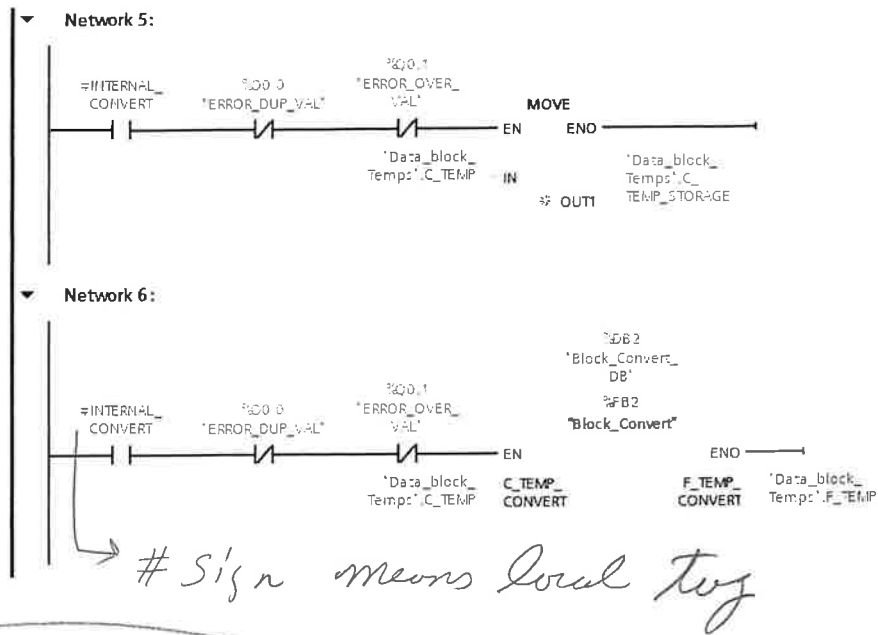
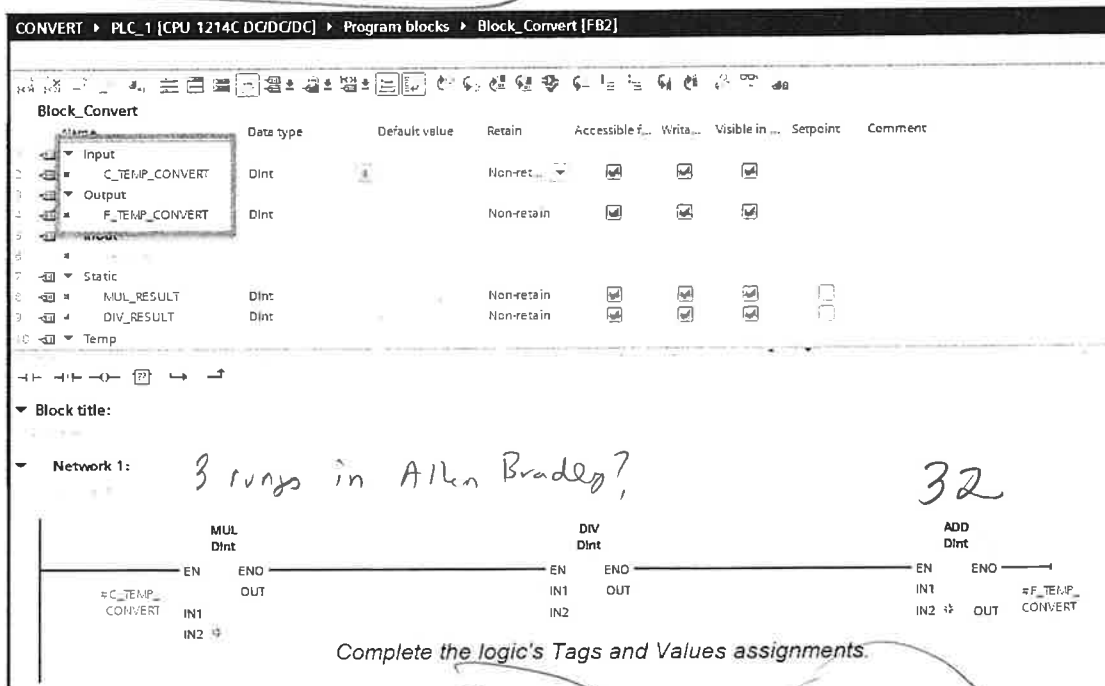




Function Block: Block_Convert



When "Block_Convert" is called, the tag C_TEMP is passed as an Input Instance to the local tag C_TEMP_CONVERT to be used as the Celsius value in the calculation.

When the calculation is completed, the local tag F_TEMP_CONVERT the resulting Fahrenheit value, is passed as an Output Instance to the tag F_TEMP.



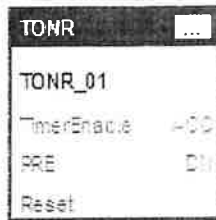
Can do same in ST SK

Shaw ↗ TWS_enabled =
↘ HMI = 0

FUNCTION BLOCK – TIMERS

Studio 5000

TONR – Timer On Reset



The **TONR** is an Energize Timer to Delay ON.

A True **TimerEnabled** pin causes the timer to begin elapsing the Accumulated value.

When the **Preset** and **Accumulated** are equal the **DN** bit pin is set.

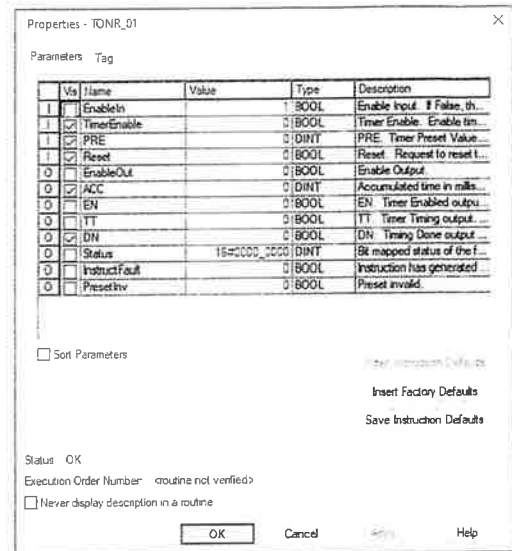
When the **TimerEnabled** is False, the Accumulated value returns to zero and the **DN** is reset.

The **Reset** is optional, when the **Reset** pin is True the Accumulated value returns to zero even if **TimerEnable** is True.

The **EN** (Enable bit), **TT** (Timer Timing Bit) and **ACC** (Accumulated Value) will be shown when checked.

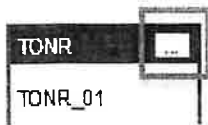
Dragging the **TONR** instruction creates in the Parameters and Local Tag the Tag Name and FBD_TIMER Data Type.

Default Properties



Example 1

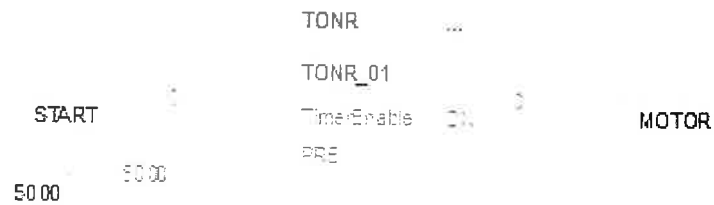
In this example the PRE (Preset) and DN (Done) are checked.



Properties button.



A START switch is turned to make the **TimerEnable** pin True. The Accumulated begins elapsing. After a 5 second delay, Preset and Accumulated are equal, the **DN** pin is set turning On the MOTOR.



The **PRE** (Preset) uses an IREF with a constant value of 5000.

Example 2



In this example the IREF to the **PRE** uses a Tag for the Preset Value.

Example 3



The Reset is checked in the Properties. If the **Reset** pin is True the Accumulated value is reset to zero and the DN (Done) pin is reset if set.

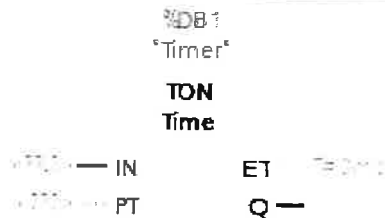
If the **TimerEnable** pin is True when the **Reset** pin is True the Accumulated Value is zeroed and will remain at zero until the **Reset** is False. Then, when the **Reset** pin is False the True **TimerEnable** will cause the Accumulated to begin elapsing.

Siemens

TON – Timer On

When the **TON** instruction is dragged onto the Network the Call dialog opens. Type in the timer's Tag **Name** and select **OK**.

The timer's data is assigned in the Project Tree to System blocks and Program resources.



Example 1

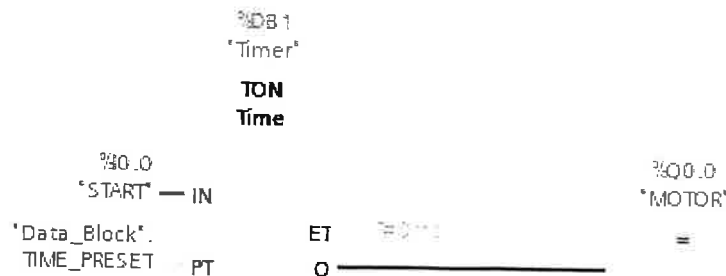
A START switch is turned to make the **In** pin True. The **ET** (Elapsed Time) begins elapsing. After a 5 second delay, **PT** (Preset Time) and **ET** are equal, the **Q** (Output) pin is set turning On the MOTOR.



To control the Motor the **Q** pin is attached to the Assign Instruction with the MOTOR Tag.

When the **In** is False, the **ET** value returns to zero and the **Q** is reset.

Example 2



In this example the **PT** uses a Tag for the Preset Value.

ANALOG INPUT EXAMPLES

Scaled Value: 0 to 1000 rpm

Bit Minimum and Maximum of Module: 0 to 32767

Bit Value from Analog Input: 16384

$$\text{Slope} = (1000 - 0) / (32767 - 0), \text{Slope} = 1000/32767 = 0.03051$$

$$\text{Offset} = 0 - (0 \times 0.031) = 0$$

$$\text{Scaled Value} = (16384 \times 0.03051) + 0 = 499.8758 \text{ rpm}$$

Scaled Value: 100° C to 500° C

Bit Minimum and Maximum of Module: 0 to 32767

Bit Value from Analog Input: 16384

$$\text{Slope} = (500 - 100) / (32767 - 0), \text{Slope} = 400/32767 = 0.01220$$

$$\text{Offset} = 100 - (0 \times 0.01120) = 100$$

$$\text{Scaled Value} = (16384 \times 0.01220) + 100 = 199.8848 + 100 = 299.8848 \text{ ° C}$$

ANALOG OUTPUT EXAMPLE

Scaled Value: 0 to 1000 rpm

Bit Minimum and Maximum of Module: 0 to 32767

Scaled Value to Analog Output: 500 rpm

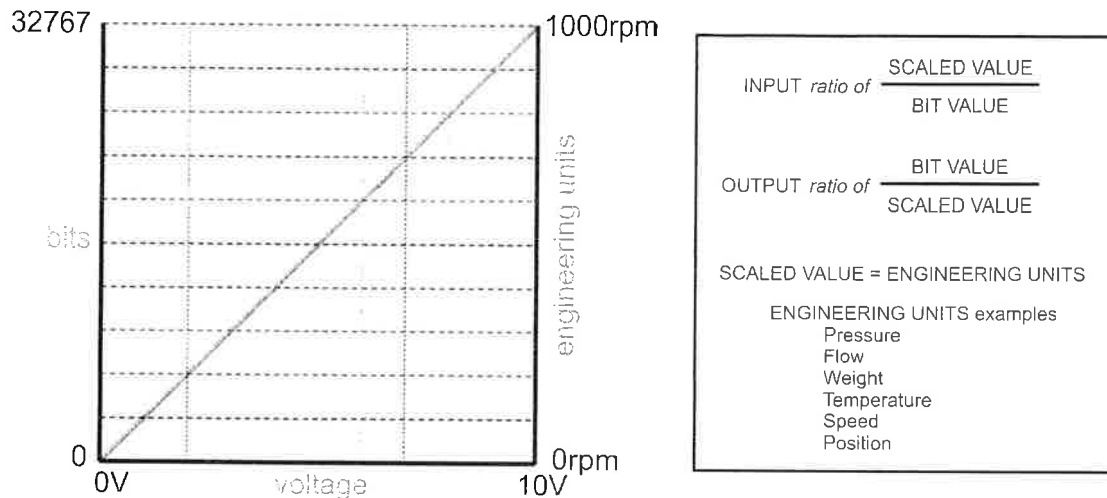
$$\text{Slope} = (32767 - 0) / (1000 - 0), \text{Slope} = 32767/1000 = 32.767$$

$$\text{Offset} = 0 - (0 \times 0.031) = 0$$

$$\text{Scaled Value} = (500 \times 32.767) + 0 = 16,383.5 \text{ [16,384 bit value]}$$

SCALING

For analog inputs and outputs, scaling is defining a ratio of the engineering units, the value range being measured, and the bit values produced by the electrical signal.



SCALING CALCULATIONS

bit min or max: minimum or maximum bit value of the module

scaled min or max: minimum or maximum value in engineering units, i.e. 0 psi. to 500 psi

scaled value: value in engineering units to determined from bit value

bit value: binary equivalent of scaled value

ANALOG INPUT (Bit value to Scaled Value)

Scaled value = (bit value x slope) + offset

Slope = (scaled max. – scaled min.) / (bit max. – bit min.)

Offset = scaled min. – (bit min. x slope)

ANALOG OUTPUT (Scaled value to Bit value)

Bit value = (scale value x slope) + offset

Slope = (bit max. – bit min.) / (scaled max. – scaled min.)

Offset = bit min. – (scaled min. x slope)

FRO → BCD_TO

TWS-input

SIEMENS STRUCTURED TEXT

A dialog opens *when* the TON instruction is dragged into the *program*, the tag name in this example is DWELL. The TON is a System block and Program resource.

Timer structure

```
"DWELL".TON(IN:= "SWITCH",
             PT:=T#5000ms);
```

Timer values Timer status bit formats

```
"DWELL".PT      "DWELL".Q
"DWELL".ET      "DWELL".IN
```

Examples:

Energize Timer Delay ON

```
"DWELL".TON(IN:= "SWITCH",
             PT:=T#5000ms);
"OUTPUT" := NOT "DWELL".Q;
```

Energize Timer Delay OFF

```
"DWELL".TON(IN:= "SWITCH",
             PT:=T#5000ms);
"OUTPUT" := NOT "DWELL".Q AND "DWELL".IN;
```

Setting Prest Time

```
"DWELL".PT := T#5000ms;
```

Format when dragged in:

```
"DWELL".TON(IN:=_bool_in_,
             PT:=_time_in_,
             Q=>_bool_out_,
             ET=>_time_out_);
```

The IN and PT are required.

If the Q and ET are unassigned, they will be deleted when pressing ENTER to complete the instruction.

Energize Timer Delay ON

```
"DWELL".TON(IN:= "SWITCH",
             PT:=T#5000ms);
IF "DWELL".Q THEN
    "OUTPUT" := 1;
    additional actions
ELSE;
    "OUTPUT" := 0;
    additional actions
END_IF;
```

Energize Timer Delay OFF

```
"DWELL".TON(IN:= "SWITCH",
             PT:=T#5000ms);
IF NOT "DWELL".Q AND "DWELL".IN THEN
    "OUTPUT" := 1;
ELSE;
    "OUTPUT" := 0;
END_IF;
```

A dialog opens *when* the R_TRIG instruction is dragged into the *program*, the tag name in this example is ONE_SHOT. The R_TRIG is a System block and Program resource.

// #PULSE is a Local Instance.

One Shot Rising structure

```
"ONE_SHOT"(CLK:="PUSHBUTTON",
            Q=>#PULSE);
MY_TAG := #PULSE
```

One shot bit used with additional conditions

```
"ONE_SHOT"(CLK:="PUSHBUTTON",
            Q=>#PULSE);
IF #PULSE AND OK_TO_RUN THEN
    MY_TAG := 1;
ELSE
    MY_TAG := 0;
END_IF;
```

Format when dragged in:

```
"ONE_SHOT"(CLK:=_bool_in_,
            Q=>_bool_out_);
```

Notice the instruction name is not shown.



Hmi-enable - disable = 0

STUDIO 5000 STRUCTURED TEXT

Tag name is DWELL using Data Type FBD_TIMER.

Format when dragged in:
TONR()

Timer structure

```
DWELL.PRE := 5000;  
DWELL.TimerEnable := SWITCH;  
TONR(DWELL);
```

Timer Values	Timer status bit formats
DWELL.PRE	DWELL.EN
DWELL.ACC	DWELL.TT
	DWELL.DN

Examples:

Energize Timer Delay ON

```
DWELL.PRE := 5000;  
DWELL.TimerEnable := SWITCH;  
TONR(DWELL);  
OUTPUT := DWELL.DN;
```

Energize Timer Delay OFF

```
DWELL.PRE := 5000;  
DWELL.TimerEnable := SWITCH;  
TONR(DWELL);  
OUTPUT := NOT DWELL.DN AND DWELL.EN;
```

Setting Preset

```
DWELL.PRE := TIME_VALUE;
```

Energize Timer Delay ON

```
DWELL.PRE := 5000;  
DWELL.TimerEnable := SWITCH;  
TONR(DWELL);  
IF DWELL.DN THEN  
    OUTPUT := 1;  
    additional actions  
ELSE;  
    OUTPUT := 0;  
    additional actions  
END_IF;
```

Energize Timer Delay OFF

```
DWELL.PRE := 5000;  
DWELL.TimerEnable := SWITCH;  
TONR(DWELL);  
IF NOT DWELL.DN AND DWELL.EN THEN  
    OUTPUT := 1;  
ELSE;  
    OUTPUT := 0;  
END_IF;
```

Tag name ONE_SHOT using Data Type FBD_ONESHOT.

One Shot Rising structure

```
ONE_SHOT.InputBit := PUSHBUTTON;  
OSRI(ONE_SHOT);
```

Format when dragged in:
OSRI()

Examples:

Format for assigning Tag to Output Bit:

```
ONE_SHOT.InputBit := PUSHBUTTON;  
OSRI(ONE_SHOT);  
MY_TAG := ONE_SHOT.OutputBit
```

One shot bit used with additional conditions

```
ONE_SHOT.InputBit := PUSHBUTTON;  
OSRI(ONE_SHOT);  
IF ONE_SHOT.OutputBit AND OK_TO_RUN THEN  
    MY_TAG := 1;  
ELSE;  
    MY_TAG := 0;
```



OAKLAND COMMUNITY COLLEGE

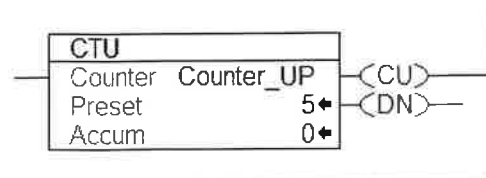
ROBOTICS, AUTOMATED SYSTEMS
TECHNOLOGY

Counter Instructions Summary

Studio 5000 COUNTERS

Counter Up (CTU)

The Counter Up (CTU) is an output instruction that will turn on a controlled output when the accumulated value is greater than or equal to the preset value.

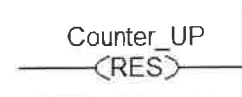


The greater than or equal to value will result in the Done bit being set. The Done bit may be used to control other logic.

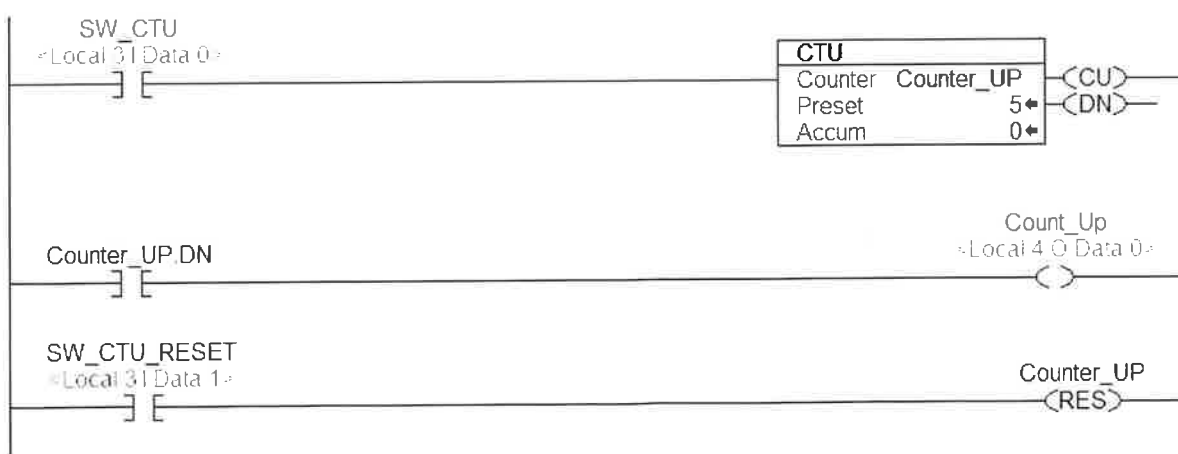
When the rung transitions from false to true, the counter accumulated value increments by one.

Counter Reset (RES)

When the rung is TRUE, the Reset (RES) instruction returns the Counter Up (CTU) with the same Tag Accumulated value to zero.



Counter Up Example

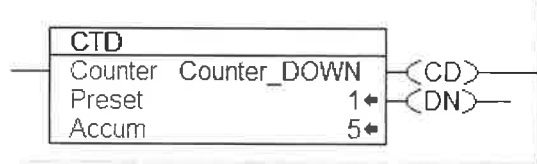


Status Bits

Counter Up (CTU)	
Counter Up Enable Bit .CU	TRUE when counter rung True
Done Bit .DN	TRUE when Accumulated (ACC) value is greater than or equal the Preset (PRE) value

Counter Down (CTD)

The Counter Down (CTD) is an output instruction that will turn on a controlled output while the accumulated value is greater than the preset value.



The accumulated value is greater or equal to the preset value will result in the Done bit being set. The Done bit may be used to control other logic.

When the rung transitions from false to true, the counter accumulated value decrements by one.

Up-Down Counter is created by combining an Up Counter with a Down Counter. Both counters use the same Counter Data Type Tag. The **.DN** status bit is controlled by the Up Counter. The counters are controlled by independent inputs.

Siemens COUNTERS

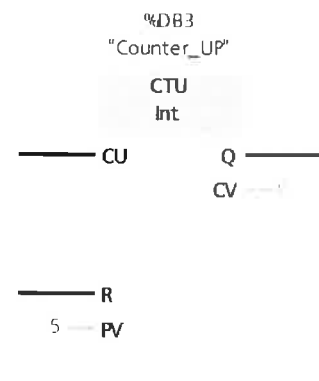
Counter Up (CTU)

The Counter Up (CTU) is an instruction that will turn on a controlled output when the Count Value (CV) is greater than or equal to the Preset Value (PV).

The equal values will result in the Output Bit (QU) being set. The Output Bit is used to control other logic. Note: **Q** shown in instruction, use **QU** when assigning Tag.

The Reset (**R**) returns the Count Value to zero.

When the rung transitions from false to true, the counter increments.



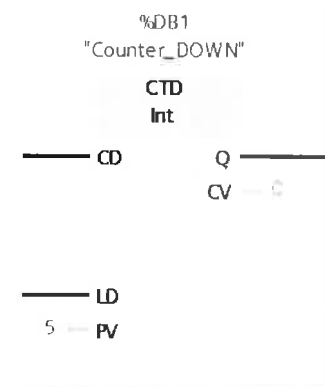
Counter Down (CTD)

The Counter Down (CTD) is an instruction that will turn on a controlled output when the Count Value (CV) is equal to or less than zero.

The equal values will result in the Output Bit (QD) being set. The Output Bit is used to control other logic. Note: **Q** shown in instruction, use **QD** when assigning Tag.

The Load (**LD**) returns the Count Value to the preset Value.

When the rung transitions from false to true, the counter decrements.

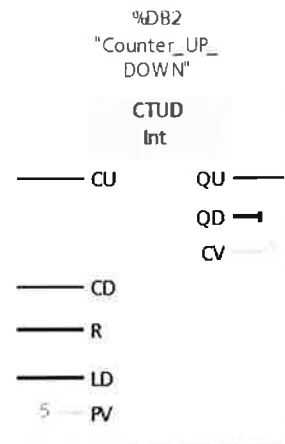


Counter Up/Down (CTUD)

The Counter Up/Down (CTUD) is an instruction that will turn on a controlled output (QU) when the Counter Value (CV) is greater than or equal to the Preset Value (PV).

A Counter Value (CV) less than or equal to the Preset Value (PV) will turn on a controlled output (QD).

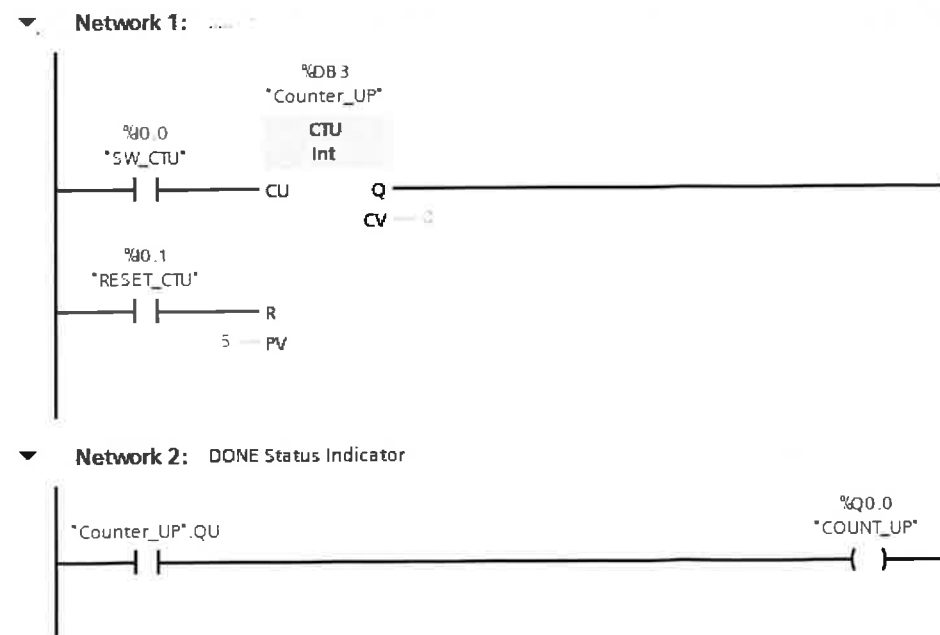
When the rung transitions from false to true, the counter increments or decrements, depending on the use of the Count Up (CU) or Count Down (CD).



Status Bits

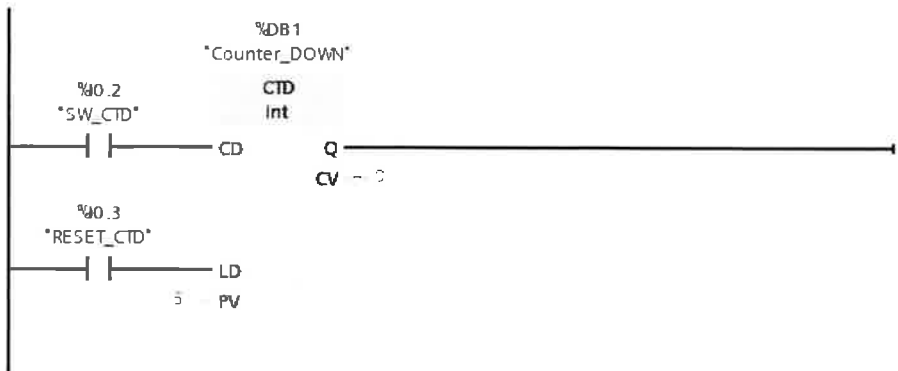
Counter Up (CTU)	
Counter Up Input Bit .CU	TRUE when counter rung True
Count Status Bit .QU	TRUE when the Count Value (CV) is greater than or equal to the Preset Value (PV).
Counter Down (CTD)	
Counter Down Input Bit .CD	TRUE when counter rung True
Count Status Bit .QD	TRUE when the Count Value (CV) is equal to or less than zero

Counter Up Example

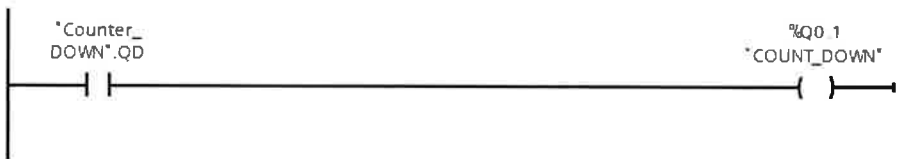


Counter Down Example

Network 3:

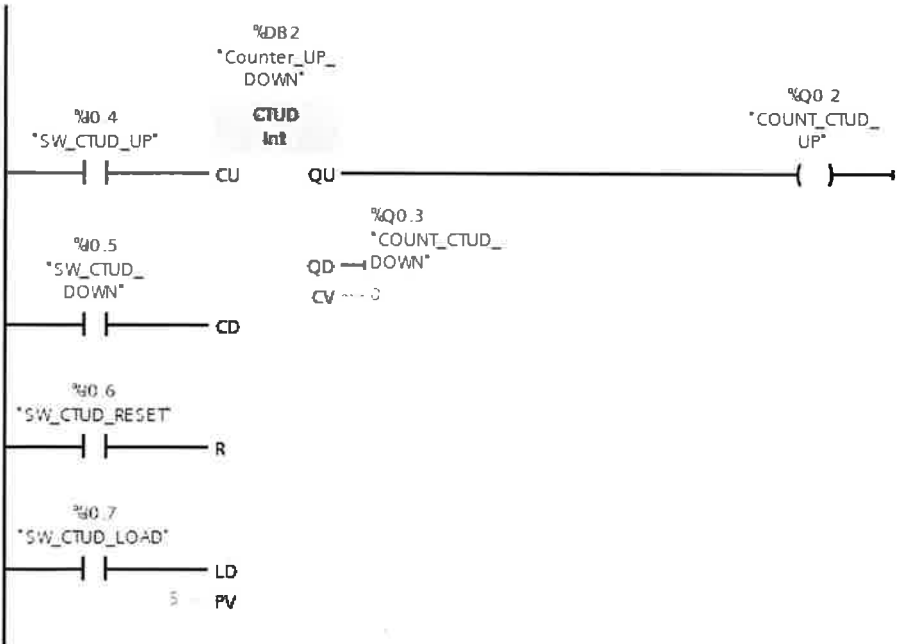


Network 4:



Counter Up/Down Example

Network 5:



LOGICAL INSTRUCTIONS "TRUTH TABLES"

The logical instructions perform the operation on all the bits in the defined structures.

Illustrated below are four bit structures.

AND

Source A / IN1	Source B / IN2	Destination/OUT
0	0	0
0	1	0
1	0	0
1	1	1

AND

$\begin{array}{cc} \text{---}11\text{---}11\text{---}(\)\text{---} \\ \text{A} \quad \text{B} \end{array}$

OR

Source A / IN1	Source B / IN2	Destination/OUT
0	0	0
0	1	1
1	0	1
1	1	1

OR

$\begin{array}{c} \text{A} \\ \text{---}11\text{---} \\ \text{B} \\ \text{---}11\text{---} \end{array} \text{---}(\)\text{---}$

XOR Exclusive OR

Source A / IN1	Source B / IN2	Destination/OUT
0	0	0
0	1	1
1	0	1
1	1	0

XOR

$\begin{array}{c} \text{A} \\ \text{---}11\text{---}11\text{---} \\ \text{B} \\ \text{---}11\text{---}11\text{---} \end{array} \text{---}(\)\text{---}$

NOT

Source / IN1	Destination/OUT
0	1
1	0

$\begin{array}{c} \text{A} \\ \text{---}11\text{---} \end{array} \text{---}(\)\text{---}$

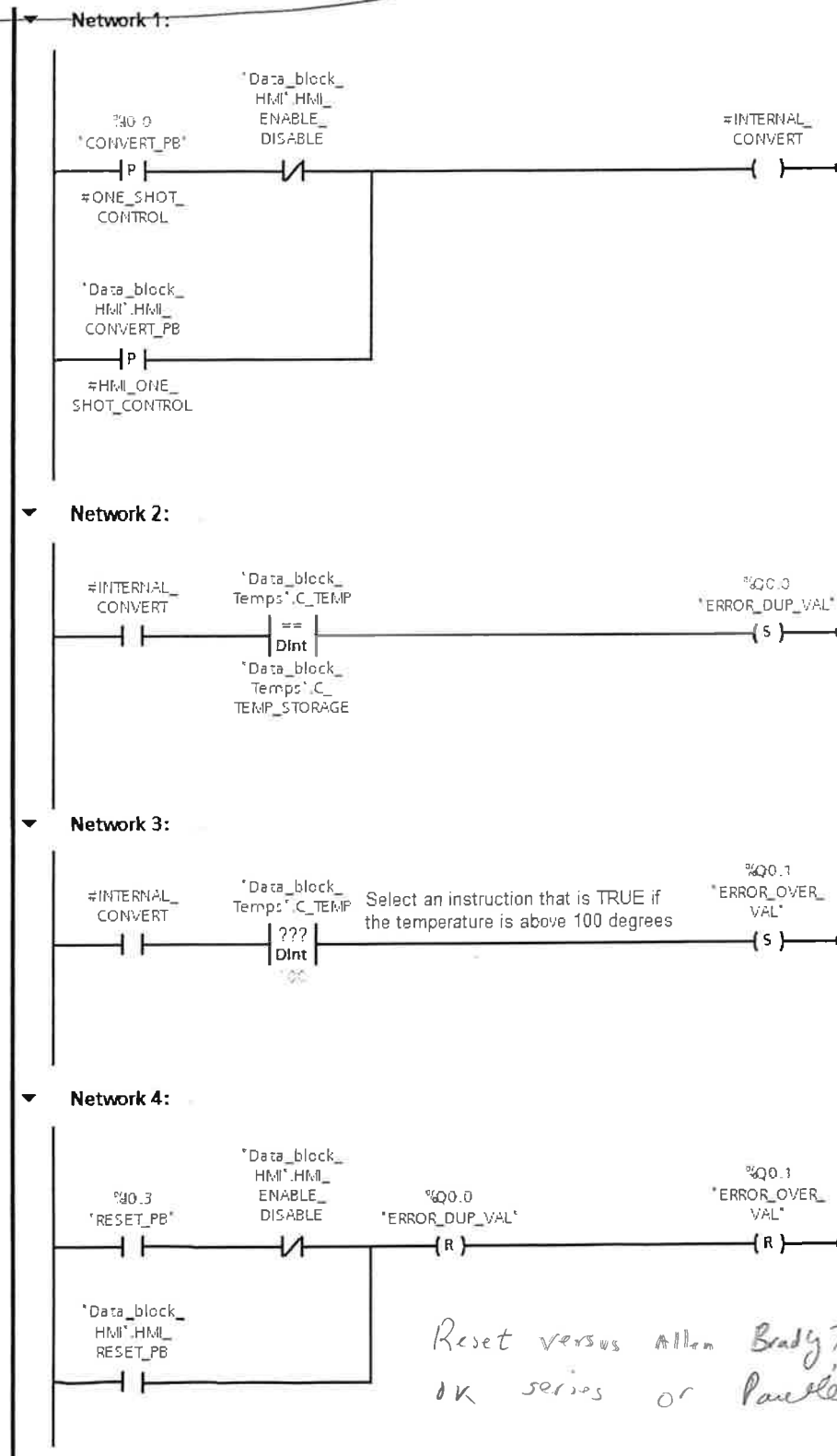
- Panypoint 17M1

Shows det

Siemens no return or input.

Siemens Temperature Conversion

Function Block: Block_Input_Control



Studio 5000 Temperature Conversion

Routine: InputControlRoutine

Buffering works because of one shot

ONLY FIRST SCAN ON

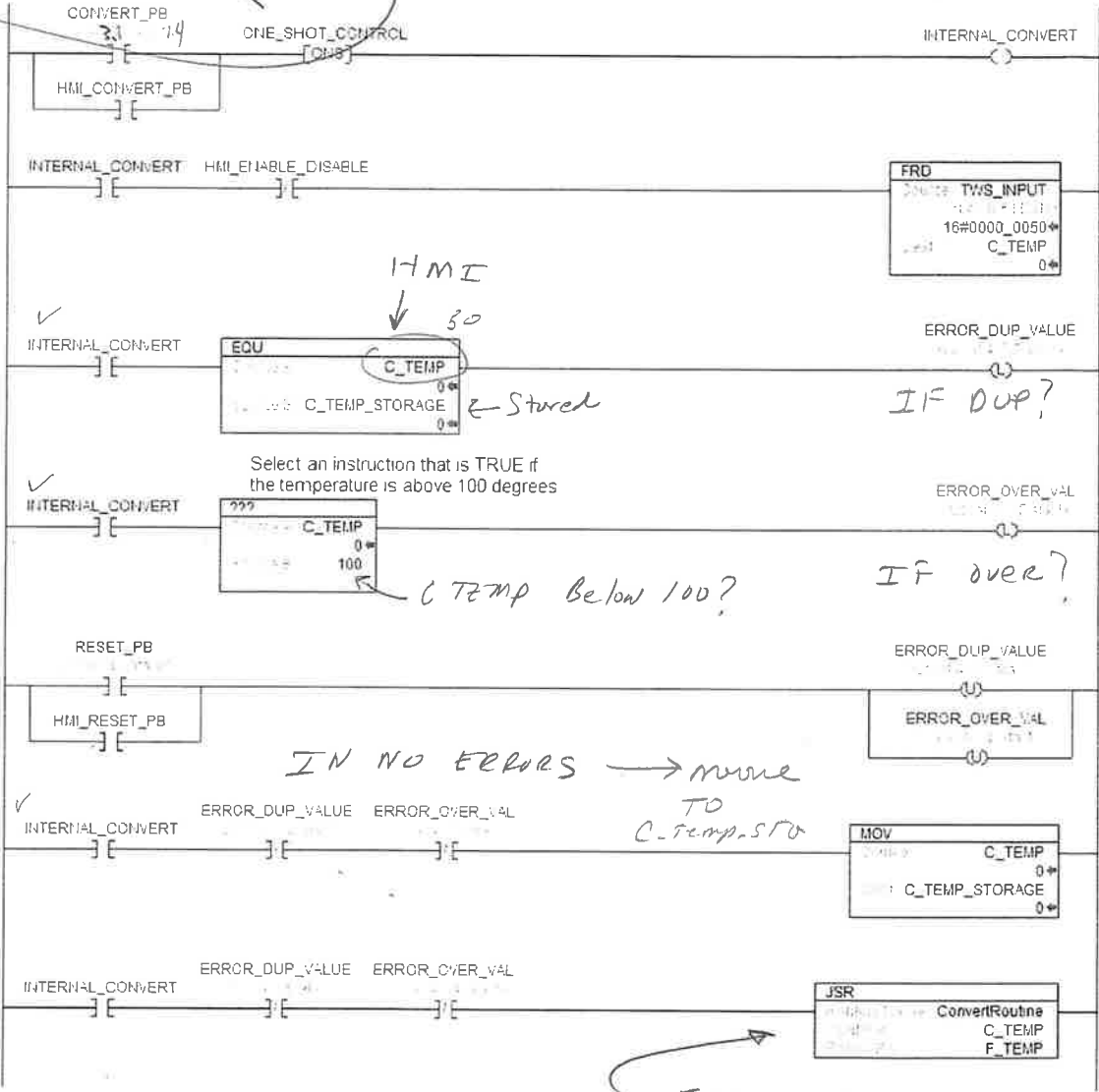
~~Temperature Conversion~~

Skipped AFTER FIRST SCAN
TRUE one scan

UNLATCH ERRORS

TRUE one scan only

① missing rung
ADD + MULT + DIV
Return



IF DUP?

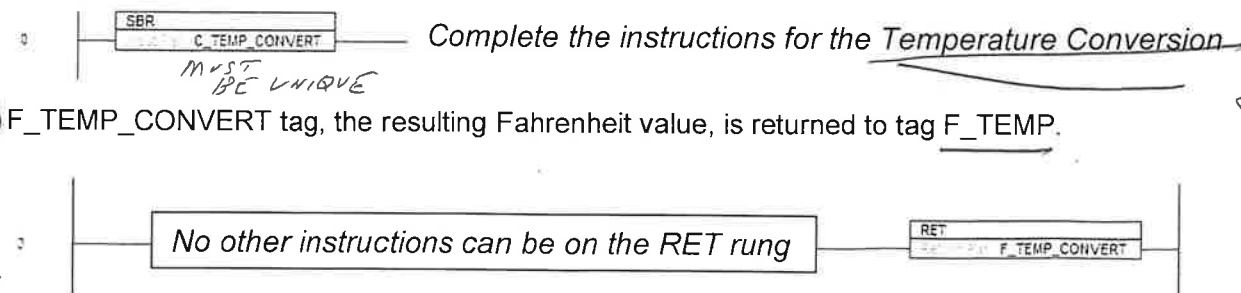
IF over?

LATCH
Keep on until Reset

Routine: ConvertRoutine

CELCIUS TO F°

The C_TEMP tag value is passed to the Tag C_TEMP_CONVERT to be used as the Celsius value in the calculation.



* ADD + SUB
- RETURN RUNG
only return instruction

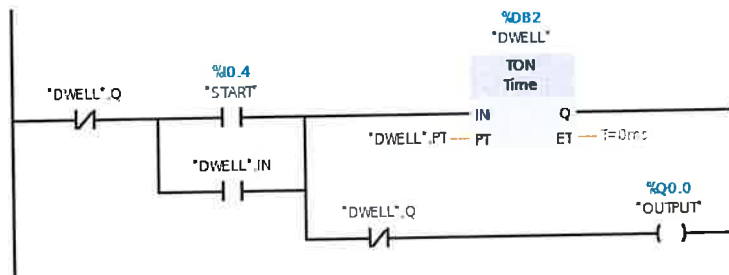
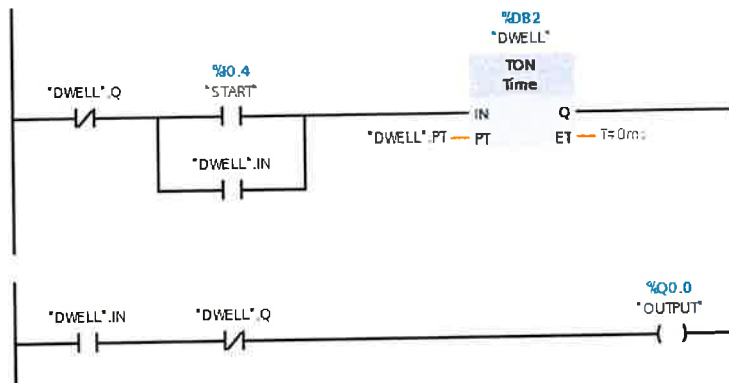


Return Back
TO F_TEMP

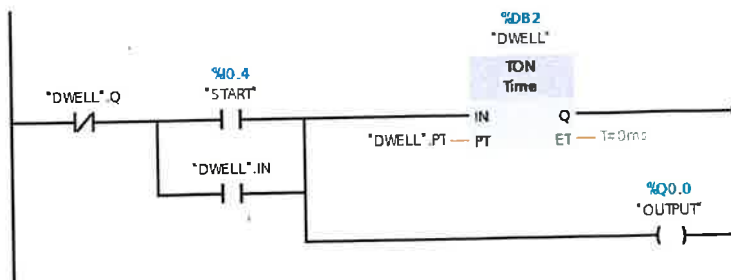
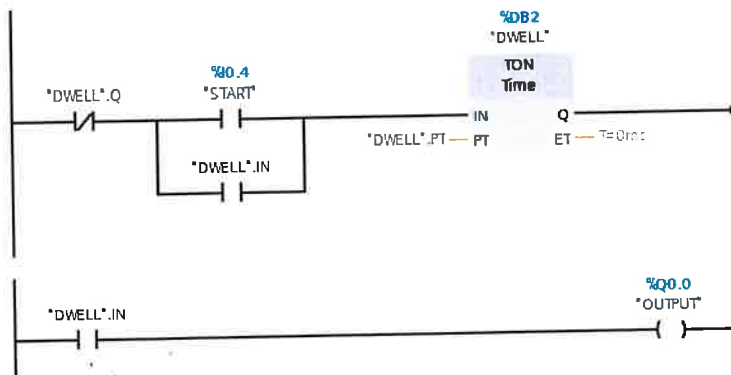
same subroutine over and over same values

MOVE TIMER RUNG EXAMPLES STUDIO SIEMENS

The following two examples are variations of the "conventional" logic structures.



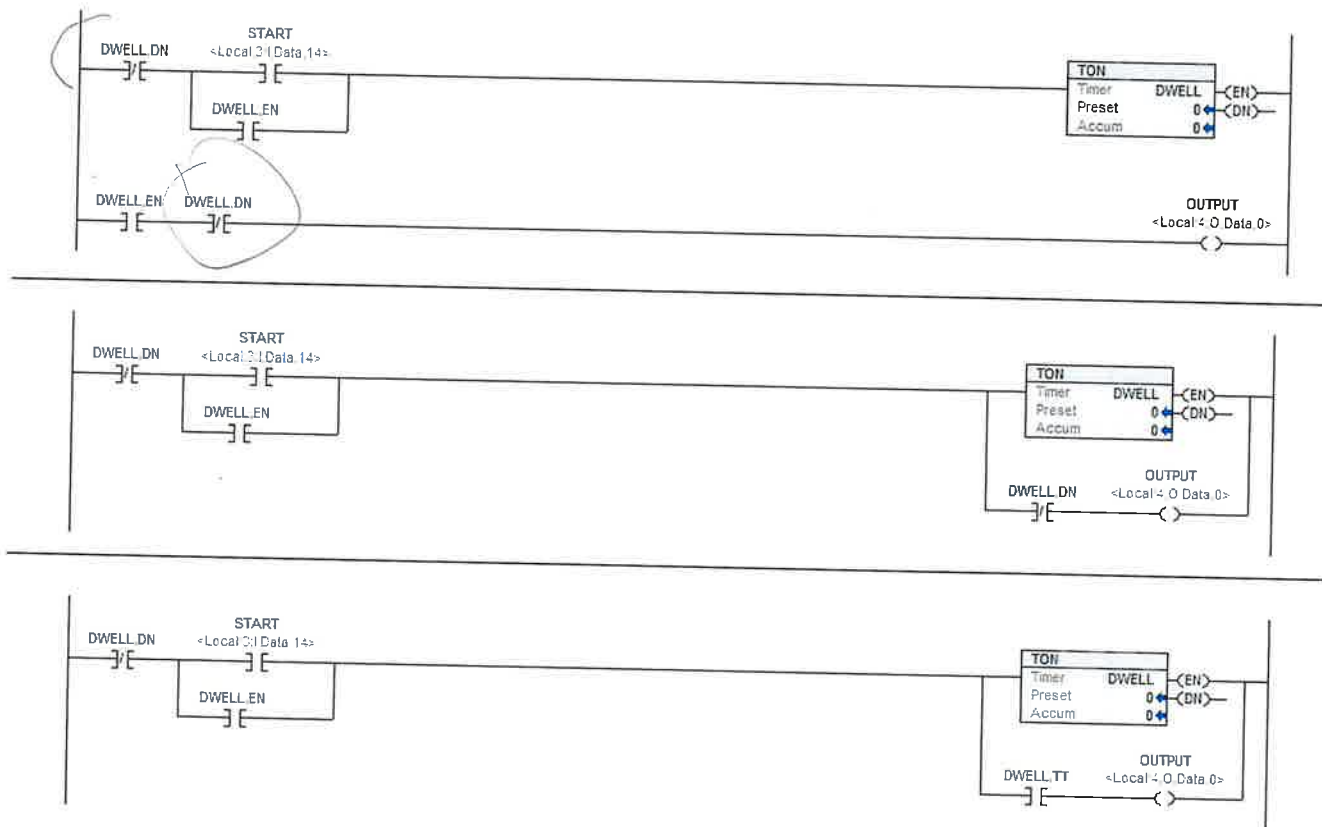
The following two examples are valid because the Timer resets itself and results in the Output turning off.



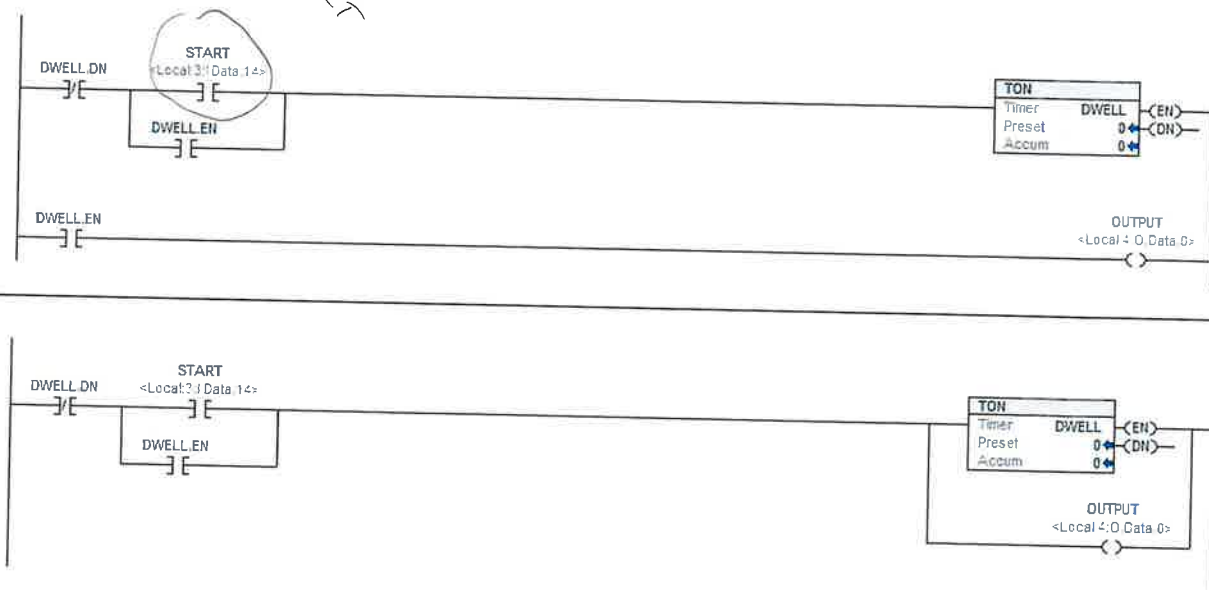
Handwritten: Hm! more hmi

MOVE TIMER RUNG EXAMPLES STUDIO 5000

The following three examples are variations of the "conventional" logic structures.



The following two examples are valid because the Timer resets itself and results in the Output turning off.



SELECT – CASE OF

CASE_OF

A **CASE_OF** selects an action based on a numerical value.

The controller executes only the statements that associated with the first matching selector Tag value.

END_CASE

After executing the action, the execution then goes to the **END_CASE** statement

Example:

```
CASE Tag1 OF
  1: Tag2 := Tag3;
  2: Tag2 := Tag4;
  3: Tag2 := Tag5;
  4: Tag2 := Tag6;
END_CASE;
```

- If **Tag1** has a value of 2, then **Tag2 := Tag4** is selected, and then execution continues to the **END_CASE**.
- If **Tag1** has a value of 4, then **Tag2 := Tag6** is executed, and then execution continues to the **END_CASE**.
- If **Tag1** does not have a value of 1 thru 4 the program execution continues to the **END_CASE**.

ELSE

An optional **ELSE** may be used to define a statement if the selectors Tag values does not match a Case.

If the **CASE** selector tag does not have one of the defined values, an **ELSE** statement is executed.

Example:

```
CASE Tag1 OF
  1: Tag2 := Tag3;
  2: Tag2 := Tag4;
  3: Tag2 := Tag5;
  4: Tag2 := Tag6;
ELSE
  Tag2 := 0;
END_CASE;
```



Example:

```
CASE Tag1 OF
  1: Tag2 := 1;
    Tag3 := 0;
    Tag4 := 0;
  2: Tag2 := 0;
    Tag3 := 1;
    Tag4 := 0;
  3: Tag2 := 0;
    Tag3 := 0;
    Tag4 := 1;
END_CASE;
```

A **CASE** action may have more than one action.

In this example the **CASE** selected will set one of the Tags and reset the other Tags.

Example:

```
CASE Tag1 OF
  1..5: Tag2 := 1;
        Tag3 := 0;
        Tag4 := 0;
  6..10: Tag2 := 0;
         Tag3 := 1;
         Tag4 := 0;
  11..15: Tag2 := 0;
          Tag3 := 0;
          Tag4 := 1;
END_CASE;
```

A **CASE** selection can use a range to determine the selected action.

In this example if the **CASE** selector Tag1 has a value of 7, then the action with the range of 6..10 is executed.

Example:

```
CASE Tag1 OF
  1..5: Tag2 := 1;
        Tag3 := 0;
        Tag4 := 0;
  6, 8: Tag2 := 0;
        Tag3 := 1;
        Tag4 := 0;
  9, 11..15: Tag2 := 0;
             Tag3 := 0;
             Tag4 := 1;
END_CASE
```

A **CASE** selection can use multiple non-consecutive values to determine the selected action.

In this example if the **CASE** selector Tag1 has a value of 6 or 8, then the action with 6,8 is executed.

Values and ranges may also be used.

